

¿Qué es React?

React es una biblioteca de JavaScript de código abierto desarrollada por Meta que se enfoca en la construcción de interfaces de usuario. Se usa en el desarrollo frontend para crear páginas web rápidas, interactivas y aplicaciones de una sola página (SPA).

¿Para qué sirve?

Su propósito principal es facilitar el desarrollo de aplicaciones web complejas, especialmente las aplicaciones de una sola página (SPA). Su objetivo es ser sencillo, declarativo y fácil de combinar. React sólo maneja la interfaz de usuario en una aplicación; React es la **Vista** en un contexto en el que se use el patrón MVC (Modelo Vista-Controlador); se encarga de actualizar la pantalla de manera eficiente sin que el usuario tenga que recargar la página del navegador de forma manual.

¿Cuáles son sus características?

Virtual DOM: Representación memoria de la interfaz real de la aplicación. Su propósito principal es agilizar drásticamente los cálculos de renderizado. Funciona de la siguiente manera: cuando cambian los datos de tu aplicación, React actualiza primero este "DOM Virtual". Luego, compara esta nueva versión con la versión anterior para calcular las diferencias exactas. Finalmente, React actualiza en el DOM real (la pantalla que ves) solo las partes exactas que han cambiado, lo que le otorga su excelente rendimiento.

JSX (JavaScript XML): Extensión de sintaxis que permite escribir código similar a HTML directamente dentro de JavaScript. Permite combinar la estructura visual de un componente y su lógica de programación en un solo lugar.

Props (Propiedades): Mecanismo que utiliza React para pasar datos de un componente a otro, específicamente de un componente "padre" a un componente "hijo". Se pueden interpretar como los argumentos o parámetros que le pasamos a una función. Las props son "solo de lectura" (el componente hijo que las recibe no puede modificarlas directamente)

Estados de los componentes (State): Memoria interna y privada del propio componente. Se utiliza para almacenar datos que pueden cambiar con el tiempo debido a la interacción del usuario. Cuando el estado de un componente cambia, React reacciona automáticamente y vuelve a renderizar ese componente para mostrar la información actualizada.

Ciclos de Vida (Lifecycle): Fases por las que un componente pasa desde que nace hasta que desaparece de la pantalla. Las etapas principales son: mounting, updating y unmounting.

¿Cuáles son sus ventajas y desventajas?

Entre sus mayores ventajas destaca su buen rendimiento gracias al Virtual DOM, ya que solo actualiza las partes exactas de la pantalla que han cambiado. La reutilización de componentes ahorra mucho tiempo de desarrollo a largo plazo, y al estar respaldado por

Meta, cuenta con una de las comunidades de desarrolladores y ecosistemas de herramientas más grandes del mundo. Por el otro lado, React tiene una curva de aprendizaje pronunciada debido a conceptos como JSX o la gestión avanzada de estados. Además, al ser solo una biblioteca visual y no un framework completo, a menudo requiere instalar, configurar y aprender herramientas de terceros para manejar aspectos como el enrutamiento o el manejo global de datos, lo que puede resultar abrumador en configuraciones iniciales.

¿Qué es React Bootstrap?

React Bootstrap es una adaptación del framework de diseño CSS "Bootstrap", pero reescrito desde cero para funcionar de manera nativa dentro del ecosistema de React. El Bootstrap tradicional suele depender de librerías como jQuery para que funcionen sus elementos interactivos (ej: menús desplegables), lo cual choca directamente con la forma en que React maneja el DOM. React Bootstrap soluciona este problema transformando todos los elementos de diseño de Bootstrap en componentes puros de React, permitiendo a los desarrolladores crear interfaces atractivas, responsivas y limpias de forma rápida sin comprometer la arquitectura de su aplicación